

The Fool's Descent - Updating a relational database in MySQL for the challenge

The database was revised to improve consistency between the conceptual model and the MySQL implementation while maintaining a normalized structure. The updated Entity Relationship Diagram (ERD) preserves most of the original design and introduces a small number of modifications to better support gameplay mechanics and future expansion. The database uses primary keys, foreign keys, and integrity constraints to ensure data consistency and reliability.

Entity Relationships and Integrity Constraints

Player and Player_Stats

The relationship between **Player** and **Player_Stats** is one-to-one (1:1). Each player has a single statistics record, and each statistics record belongs to one player.

Integrity Constraints

- Player.user_id is the primary key.
- Player_Stats.user_id is a foreign key referencing Player.user_id.
- The foreign key is unique to guarantee that each player has only one statistics record.

Player and Current_Run

The relationship between **Player** and **Current_Run** is one-to-many (1:N). A player can complete multiple runs, while each run belongs to one player.

Integrity Constraints

- Current_Run.user_id is a foreign key referencing Player.user_id.

Current_Run and Run_Enemy_Encounters

The relationship between **Current_Run** and **Run_Enemy_Encounters** is one-to-many (1:N). A run may contain multiple enemy encounters, while each encounter belongs to a single run.

Integrity Constraints

- Run_Enemy_Encounters.run_id is a foreign key referencing Current_Run.run_id.

Enemy and Run_Enemy_Encounters

The relationship between **Enemy** and **Run_Enemy_Encounters** is one-to-many (1:N). An enemy may appear in many encounters across different runs, while each encounter references one enemy.

Integrity Constraints

- `Run_Enemy_Encounters.enemy_id` is a foreign key referencing `Enemy.enemy_id`.

Run_Enemy_Encounters and Great_Deck

The relationship between **Run_Enemy_Encounters** and **Great_Deck** is one-to-one (1:1). Each encounter is associated with one generated deck, and each generated deck belongs to one encounter.

Integrity Constraints

- `Great_Deck.encounter_id` is a foreign key referencing `Run_Enemy_Encounters.encounter_id`.

Player and Player_Deck

The relationship between **Player** and **Player_Deck** is one-to-many (1:N). A player may own multiple cards, while each deck record belongs to one player.

Integrity Constraints

- `Player_Deck.user_id` is a foreign key referencing `Player.user_id`.

Cards and Player_Deck

The relationship between **Cards** and **Player_Deck** is one-to-many (1:N). A card can appear in multiple player decks, while each deck record references a single card.

Integrity Constraints

- `Player_Deck.card_id` is a foreign key referencing `Cards.card_id`.

Player and Game_SaveState

The relationship between **Player** and **Game_SaveState** is one-to-many (1:N). A player may create multiple save states, while each save state belongs to one player.

Integrity Constraints

- `Game_SaveState.user_id` is a foreign key referencing `Player.user_id`.

Enemy and Game_SaveState

The relationship between **Enemy** and **Game_SaveState** is one-to-many (1:N). Multiple save states may reference the same enemy, while each save state stores only one current enemy.

Integrity Constraints

- `Game_SaveState.current_enemy_id` is a foreign key referencing `Enemy.enemy_id`.

Game_SaveState and PlayerUpgrade

The relationship between **Game_SaveState** and **PlayerUpgrade** is one-to-many (1:N). A save state may contain multiple upgrades, while each upgrade record belongs to one save state.

Integrity Constraints

- `PlayerUpgrade.gameSave_id` is a foreign key referencing `Game_SaveState.gameSave_id`.

Upgrades and PlayerUpgrade

The relationship between **Upgrades** and **PlayerUpgrade** is one-to-many (1:N). An upgrade may be acquired multiple times by different players, while each `PlayerUpgrade` record references one upgrade.

Integrity Constraints

- `PlayerUpgrade.upgrade_id` is a foreign key referencing `Upgrades.upgrade_id`.

Primary Key and Foreign Key Decisions

Primary keys were assigned to every entity using unique identifiers such as `user_id`, `run_id`, `enemy_id`, `card_id`, and `upgrade_id`. These keys guarantee that every record can be uniquely identified and simplify relationships between tables.

Foreign keys were created whenever an entity depends on another entity for its existence. For example, a run cannot exist without a player, an encounter cannot exist without both a run and an enemy, and a deck entry cannot exist without a valid card. These relationships enforce referential integrity and prevent inconsistent or orphaned records.

Additional constraints were included where appropriate. Usernames, card names, enemy names, and upgrade names are unique, ensuring that duplicate game assets cannot be created. Foreign key constraints and cascading operations help maintain consistency throughout the database.

Database Normalization

The schema satisfies **Third Normal Form (3NF)**.

The design satisfies **First Normal Form (1NF)** because all attributes contain atomic values and each table has a primary key.

The design satisfies **Second Normal Form (2NF)** because all non-key attributes depend entirely on their table's primary key, eliminating partial dependencies.

The design satisfies **Third Normal Form (3NF)** because non-key attributes depend only on the primary key and not on other non-key attributes. Information about players, runs, enemies, cards, upgrades, encounters, and save states is stored in separate entities, reducing redundancy and preventing update anomalies.

Justification of Changes

The updated ERD maintains the overall structure of the previous design while introducing refinements that better support the game's mechanics.

The most important addition is the **Great_Deck** entity, which stores the generated deck associated with a specific enemy encounter. Separating this information from other gameplay entities improves organization and allows future expansion without modifying existing tables.

The **Run_Enemy_Encounters** entity was also expanded with additional attributes related to encounter performance, including information such as rewards, cards played, encounter duration, and references to generated decks. These additions provide more detailed gameplay data while preserving normalization.

Relationship cardinalities were reviewed and adjusted where necessary to ensure that the conceptual model matches the foreign key structure implemented in the MySQL database. This improves consistency between the ERD and the actual schema.